ML 24/25-09 Semantic Similarity Analysis of Textual Data

Nishta Muraleedharan
nishta.muraleedharan@stud.fra-uas.de

Abstract*—Traditional methods for analyzing textual data relied on keyword matching to assess semantic similarity. However, with the advent of large-scale natural language processing (NLP) models, it is now possible to represent textual data as high-dimensional embeddings, enabling a more nuanced understanding of semantic relationships. While similarity metrics such as cosine similarity effectively compare words, phrases, and short texts, they struggle to distinguish between documents with similar contextual backgrounds when searching for specific, fine-grained information. This paper presents a novel approach to addressing this limitation by leveraging an Exponentially Weighted Mean Similarity method. Additionally, it introduces a .NET-based application designed to perform semantic similarity analysis across various levels of textual data, including words, phrases, and documents. The proposed system enhances document differentiation by capturing both overall context and detailed information, making it particularly useful for querying niche subtopics within extensive textual datasets.

***Keywords*— embeddings, cosine similarity, semantic similarity, text comparison, document analysis**

I. INTRODUCTION

Understanding the semantic similarity between textual data plays a critical task in various domains such as text classification, recommendation systems, question answering systems, automated document analysis and so on. Text representation methods have evolved tremendously over the last few decades, starting in the 1950s with count-based embeddings like Bag of Words and Term Frequency-Inverse Document Frequency (TF-IDF) [1].These methods measure similarity based on word frequency and overlap. While they work well for simple keyword-based retrieval, they struggle with synonyms, paraphrased content, and contextual relationships. The introduction of Static dense word embeddings such as Word2Vec and GloVe, improved similarity measurement by mapping words into a shared vector space, capturing some semantic relationships. However, these embeddings assign a fixed representation to words, disregarding the influence of surrounding context. More recently, transformer-based contextualized embedding models like Bidirectional Encoder Representations from Transformers (BERT) and large language model (LLM)-based embeddings (e.g., OpenAI's Ada) have provided even richer and more flexible representations by dynamically adjusting to the context in which words or phrases appear.

Embedding-based semantic analysis is widely used across various industries to improve search relevance, recommendations, and information retrieval. Some relevant examples are search engines like Google and Bing utilizing embeddings to understand query intent beyond exact keywords. E-commerce giants like Amazon and eBay enhance product ranking and recommendations using embedding models. Industries like healthcare and legal research employ them for document search and case law matching. Additionally, job portals like LinkedIn and Indeed use embeddings to align resumes with job listings based on skills and experience rather than exact keyword matches. These real-world applications highlight the effectiveness of embeddings in capturing semantic relationships, making them a powerful tool for advanced text similarity analysis.

This paper aims to develop a unified tool that utilizes embeddings to compare the semantic similarity of textual data at various levels. It must support comparisons between individual words, phrases, words or phrases with documents, and full document-to-document comparison. Large Language Model (LLM)-based embedding models, specifically OpenAI's embedding models, are used for generating embeddings because of their ability to generate fixed-size vector representations that effectively capture semantic meaning across different text lengths. Unlike models such as BERT, which are optimized for sentence-level and short-text comparisons due to their bidirectional architecture and token length constraints, OpenAI's embeddings are designed for broader semantic search tasks. They can efficiently process both short and long-form text while maintaining consistent vector dimensions, making them well-suited for similarity analysis across varying text lengths. Additionally, OpenAI's embeddings are trained on vast and diverse datasets, allowing them to generalize better across different domains, making them an optimal choice for this application.

A large document often covers multiple subtopics, making it difficult for a single embedding to capture all the nuances of its content [2]. It can effectively identify and differentiate documents based on context. However, when comparing documents against specific queries—especially in cases where all documents and the query itself share a similar overall context—standard method of computing the

cosine similarity of the two vectors struggle to differentiate which document contains the most relevant information. This poses a significant challenge in applications requiring fine-grained document comparison. For example, in legal document analysis, multiple case law documents may discuss "intellectual property rights," but a specific query might seek cases related to "copyright infringement in digital media." Since all documents discuss intellectual property, standard embedding comparisons might not effectively differentiate the one that specifically addresses digital copyright cases. Traditional similarity metrics may recognize that all papers are closely related to the query as the context is highly similar but fail to pinpoint the one most aligned with the topic of interest. This highlights the need for a more precise method to analyze and compare to documents.

To address this challenge, this paper introduces a refined approach for document similarity analysis by document chunking and using exponentially weighted mean similarity metric. The proposed method amplifies the presence of highly relevant subtopics while diminishing the impact of less relevant ones while also preserving the contextual alignment. By applying an exponential weighting function to the similarity scores of smaller textual segments within documents, the method ensures that documents with more detailed and specific matches receive a higher similarity score, even when the overall contextual similarity among all documents is high. This enables more granular differentiation between documents that share a broad thematic context but vary in their relevance to specific queries.

Additionally, this paper presents the development of a unified application that facilitates semantic similarity analysis across multiple levels of textual data. The application provides a comprehensive framework to compare words, phrases, phrases with documents, and documents with documents, integrating the exponentially weighted mean similarity method to enhance fine-grained document differentiation. Analysis is done to identify the best OpenAI embedding model, and similarity metric for better accuracy. This solution not only improves the precision of semantic similarity analysis of documents but also enables a more adaptable and scalable solution for real-world applications where contextual differentiation within a shared domain is critical.

II. METHODS

A. System Architecture Overview

The proposed application is built in C# using .NET and Visual Studio. It leverages OpenAI's embedding models for vectorization of text. A python application is also developed for interactive visualization of results. The system architecture of the text similarity analysis application consists of multiple interconnected components that work together to process textual input, generate embeddings, compute similarity scores, and visualize results.

The application accepts a set of query texts and reference texts and computes the similarity between each query-reference pair. The results are output in CSV format, as illustrated in Table 1.

TABLE I. SAMPLE CSV OUTPUT FROM THE APPLICATION

Q\R	***Reference text 1***	***Reference text 2***	***Reference text 3***
Query text 1	s_{11}	s_{12}	s_{13}
Query text 2	s_{21}	s_{22}	s_{23}

[a.] S_{ij} is the similarity between query text i and reference text j

Both query text and reference text can be words, phrases, or documents. Words and phrases are processed similarly and are thus treated alike. When the input consists of words or phrases, they are provided in a text file, with each word or phrase on a new line. The text itself serves as the label in the similarity table. In the case of documents, they are supplied as individual PDF or text files within a designated folder. The document file names are used as labels in the similarity table.

The architecture follows a structured pipeline, ensuring flexibility to compare text data at various levels (words, phrases, and documents). The overall system flow is represented by the flow diagram in Fig 1. The functioning of the major components is explained the following section.

B. Key Modules

The system consists of the following major components:

1) Input Handling: Users specify the intended mode of operation and input paths via the configuration file (appsettings.json) or command-line arguments. The application supports three operational modes:

a) Word/Phrase-to-Word/Phrase Comparison:

In this mode of operation, both query and reference texts consist of individual words or phrases. Expected inputs are paths to two text files, one containing query words/phrases and another with reference words/phrases with each word/phrase on a new line. The input handler verifies that both files are valid, non-empty text files before processing.

a) Word/Phrase-to-Document Comparison:

In this mode, query texts are documents compared against reference words or phrases. Expected inputs are paths to a folder containing query documents (PDF or text files) and a text file containing reference words/phrases (each word/phrase on a separate line). The input handler validates that the input paths are valid and contains non-empty query documents and reference word/phrase text file before reading the input. The input pdf files are converted to strings using a NuGet package called UglyToad.PdfPig. It was chosen for its lightweight, pure C# implementation with no

external dependencies, actively maintained nature, and its efficiency in reading and parsing PDF files quickly.

b) Document-to-Document Comparison:

In this mode, query documents are compared against reference documents. Expected inputs are paths to two folders—one containing query documents and the other containing reference documents (PDF or text files). The input handler ensures that the provided folders contain document files in the expected format before reading.

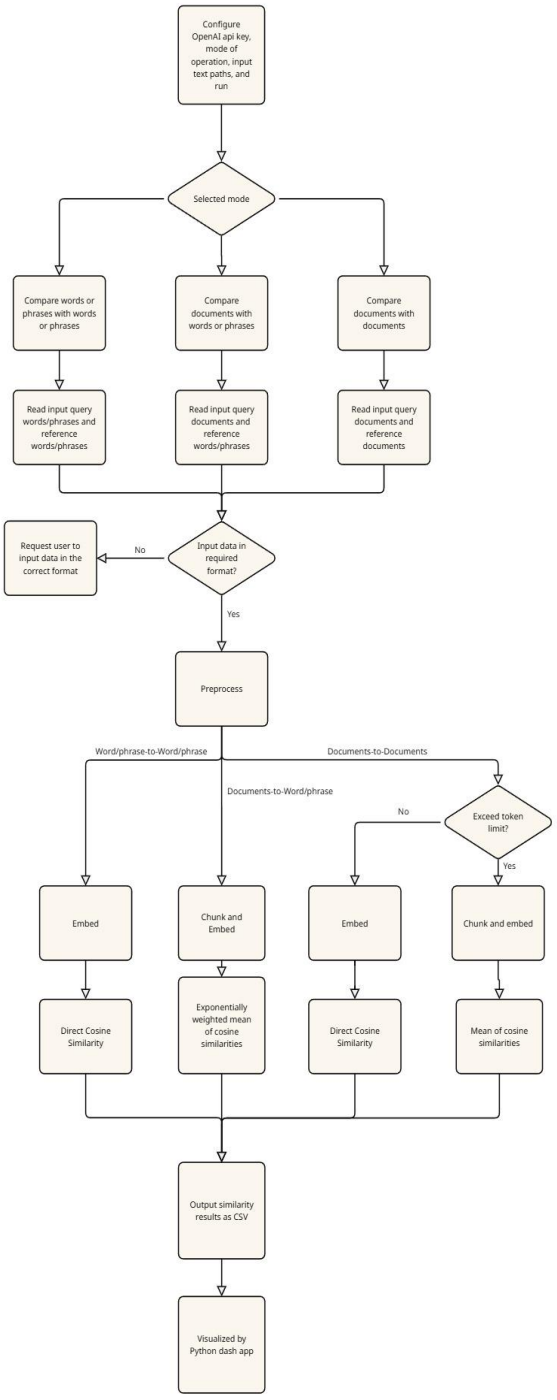

Fig. 1: System Architecture Flowchart depicting the overall workflow of the proposed application

2) Preprocessing: The preprocessing module ensures that input data is properly formatted before generating embeddings. For words and phrases, preprocessing involves trimming whitespace to standardize the input. Documents undergo more extensive cleaning, including the removal of unnecessary elements such as extra spaces, email addresses, and URLs. This reduces the total number of tokens in the text to be embedded and hence saves on computational cost. Additionally, documents are chunked based on the mode of comparison. When comparing documents with words or phrases, documents are chunked into smaller paragraphs before embedding. This is to ensure that smaller segments of text can be analyzed independently to extract smaller details, allowing more fine-grained comparison. As the application is designed to be able to process documents of different formats and domains, semantic chunking is not used, instead chunk size is selected based on the average paragraph length in English.

When comparing documents with other documents, the focus is to identify documents that are similar in terms of overall contextual alignment rather than searching for specific details. From result graphs in Listing 1 it was inferred that single embeddings for the whole document are very effective for capturing the general context of a document. Therefore, chunking is only applied when a document exceeds the maximum token limit of the embedding model. The number of tokens is counted using the NuGet package Tiktoken. This preprocessing step ensures that the application can also process long documents for semantic similarity analysis.

3) Embedding Generation: Using OpenAI's embedding model, each preprocessed input text is transformed into a high-dimensional vector representation. This approach ensures that words, phrases, and entire documents are mapped into the same vector space, enabling meaningful similarity comparisons across different text levels. For words and phrases, each input is embedded separately, maintaining their individual semantic representation. In the case of documents, each chunk is embedded independently to preserve contextual information across sections. Additionally, a comparative analysis of various OpenAI embedding models was conducted to determine the most effective model for this application, ensuring optimal performance in capturing semantic similarity.

4) Similarity Calculation: Semantic relationships between textual data are inferred by computing the similarity between their embedding representations. Since embeddings encode the meaning of words, phrases, and documents into a shared high-dimensional vector space, texts with higher semantic similarity are positioned closer to each other within this space. Various similarity metrics can be employed to quantify this closeness. In this study, multiple widely used similarity metrics were analyzed to determine their effectiveness across different levels of text

comparisons. The selected similarity metrics are described below:

a) Cosine Similarity: Cosine similarity measures the cosine of the angle between two embedding vectors. It captures the orientation rather than the magnitude. Given two vectors A and B, cosine similarity is computed as:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\|\|B\|}$$

Where A and B represents the dot product of the two vectors, and $|A|$ and $|B|$ represent their magnitudes.

b) Dot Product Similarity: The dot product directly computes the sum of the element-wise multiplication of two vectors. Unlike cosine similarity, it considers both the magnitude and direction of the vectors, making it useful in scenarios where vector magnitude carries meaningful information about textual importance. It is computed as:

$$\text{Dot Product} = \Sigma A_i B_i$$

where A_i and B_i are corresponding elements of the embedding vectors. While dot product similarity is computationally efficient, it may not always be ideal for normalization-independent similarity comparisons.

c) Euclidean Distance: Euclidean distance measures the straight-line distance between two vectors in the embedding space. It is computed as:

$$\text{Euclidean Distance} = \sqrt{\Sigma (A_i - B_i)^2}$$

where A_i and B_i are elements of the respective vectors. A lower Euclidean distance indicates higher similarity. However, since Euclidean distance is sensitive to vector magnitude, it may not be as effective as cosine similarity in cases where normalization is required.

d) Jaccard Similarity: Jaccard similarity measures the overlap between two sets and is commonly used for comparing text based on word or token overlap. It is computed as:

$$\text{Jaccard Similarity} = \frac{|A \cap B|}{|A \cup B|}$$

where *A* and *B* represent the sets of unique elements (such as words or n-grams) in two texts. Jaccard similarity is particularly useful for comparing short phrases or documents where exact word overlap is a relevant factor. However, it may not be as effective for embeddings, which capture semantic meaning beyond direct word matches. From analyzing results, it was found that cosine similarity is the most accurate similarity metric

The approach used to calculate similarity for different modes of comparison is explained below:

e) Direct Cosine similarity: For word or phrase-level comparisons, cosine similarity is computed directly between the embeddings of the query and reference text. Since single embeddings effectively capture the semantic meaning of individual words and short phrases, no additional processing is required beyond embedding generation.

Similarly, in document-to-document comparison, the objective is to identify documents with similar overall content and contextual alignment. A single embedding representation is generally sufficient for capturing the broad thematic structure of a document. Therefore, cosine similarity is calculated directly between the full document embeddings. However, when a document exceeds the maximum token limit of the embedding model, it is first divided into smaller chunks. Each chunk is embedded separately, and the final document representation is obtained by computing the mean of its chunk embeddings. This ensures that even large documents can be effectively represented while preserving their overall contextual meaning.

f) Exponentially Weighted Mean of Cosine Similarities: As mentioned earlier, when comparing query documents with specific reference words or phrases, the goal is to identify the document that contains the most relevant information about the given reference word or phrase. A direct similarity calculation between a document and a word/phrase embedding is often insufficient for this purpose, particularly when all documents and the reference share a similar overall context. Single embeddings effectively capture a document's general theme but struggle to highlight fine-grained details, making it difficult to distinguish which document is most relevant to a specific query.

To address this limitation, an exponentially weighted similarity approach is introduced. Each document is first divided into smaller chunks, and each chunk is embedded separately to retain finer details. Cosine similarity is then computed between the reference word/phrase and each document chunk, generating an array of similarity scores for each document-word/phrase pair. Since higher similarity values indicate presence of chunks with highly relevant data, a weighted mean similarity score is calculated, to ensure that chunks with higher similarity score contribute more towards the final similarity score than chunks with lower similarity score. This ensures that documents with highly relevant sections are ranked more accurately, allowing for more precise differentiation even in cases where overall document contexts are similar.

To illustrate the effectiveness of the proposed approach, consider a scenario where two query documents, Document 1 is a 1175-word essay on global warming, but has one paragraph of 147 words of relevant information about football and Document 2 is a 1000 words essay on sports but has no relevant information about football. These two documents are compared against a single reference word — Football.

From results in Listing 1 it was observed that while using OpenAI's **text-embedding-3-large** model and cosine similarity for comparing documents and words/phrases, the scores can be approximately interpreted as follows:

- **0.0 - 0.25** indicates irrelevant content.
- **0.25 - 0.35** suggests contextual alignment but no directly relevant information.
- **0.35 - 1** signifies highly relevant information.

When using the traditional approach of converting each document into single embedding vectors and calculating cosine similarity, the similarity scores are:

- Document 1: 0.33
- Document 2: 0.31

The application is not able to clearly differentiate that document 1 has significant information about football while document 2 has none. In this scenario, major parts of documents 1 and 2 are contextually different (global warming and football). But when both the documents are of similar context the differentiation becomes more insignificant.

To implement the proposed solution each document is first divided into smaller chunks of 300 words, embedded separately, and their cosine similarity with the reference word is computed. The resulting similarity scores for each document-query pair are as follows:

- Document 1: [0.11, 0.03, 0.39, 0.10]
- Document 2: [0.30, 0.28, 0.31, 0.33]

It can be observed that Document 2 maintains consistent contextual alignment with reference word but has no relevant information about it. However, the third chunk in Document 1 contains a section with highly relevant information with a similarity score of 0.39, therefore document 1 is of more interest and it should get a higher similarity score. A simple arithmetic mean of these scores results in 0.15 for Document 1 and 0.30 for Document 2, failing to identify the relevant data in document 1. To address this, an **exponentially weighted mean function** is applied:

$$S_{ewm} = \frac{\sum_{i=}^{n} w_i s_i}{\sum_{i=1}^{n} w_i}$$

Where S_{ewm} is the exponentially weighted similarity, n is the total number of chunks, s_i represents individual similarity scores, $w_i = e^{\beta s_i}$ is the weight which assigns exponentially increasing values to higher similarity values, with β controlling the bias toward higher scores.

For β = 30, the computed exponentially weighted mean scores are:

- Document 1: 0.39
- Document 2: 0.31

Unlike the simple mean, this approach assigns a higher similarity score to Document 1, accurately reflecting the presence of relevant content. The β parameter can be adjusted to control the weighting effect: higher β values emphasize the influence of highly relevant data, ensuring that the final similarity score prioritizes relevant content, while lower β values allow the score to incorporate the impact of both relevant and irrelevant data, providing a more balanced representation of overall document alignment.

The optimal chunk size and weighting parameter β were determined to be 300 words and 30, respectively. The chunk size was chosen based on the average paragraph length in English (typically ranging from 300 to 500 words). The exact value of number of words per chunk and the value β was optimized through iterative trial and error to achieve clear differentiation between relevant and non-relevant document while not ignoring the contribution of non-relevant chunks towards final score to have a balanced approach.

5) Visualization Module: A Python Dash application is used to visualize the similarity results through an interactive web interface. The similarity table generated by the C# application, as described in Table 1, is processed by the Dash app and represented using bar graphs and scatter plots for intuitive analysis. Users can choose filtered views of the reference and query inputs for clearer understanding. Additionally, individual embedding vectors are visualized, with the index values on the x-axis and vector components ranging from -1 to +1 on the y-axis. This visualization module enhances interpretability, allowing users to better analyze similarity relationships between texts.

III. Results

A. Performance of OpenAI Embedding Models

To determine the most effective embedding model for the application, multiple OpenAI embedding models were evaluated, including text-embedding-ada-002, text-embedding-small-3 and text-embedding-large-3. Each model was tested on different text comparison tasks: word-to-word, phrase-to-document, and document-to-document similarity. Cosine similarity was used as the similarity metric. The performance for Word/Phrase-to-Word/Phrase Comparison is discussed further.

Figures 2, 3, and 4 illustrate the performance of the three OpenAI embedding models, text-embedding-large-3, text-embedding-small-3 and text-embedding-ada-002 respectively when tested using a set of query and reference words/phrases. The similarity scores were computed using cosine similarity, and the results demonstrate variations in how each model captures semantic relationships.

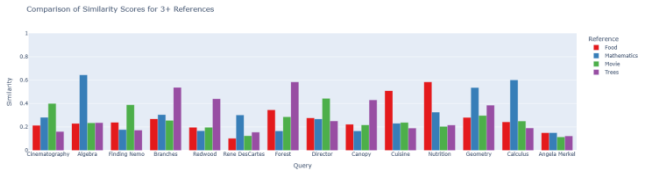

Fig 2: Performance of text-embedding-3-large while comparing words

Fig 3: Performance of text-embedding-3-small while comparing words

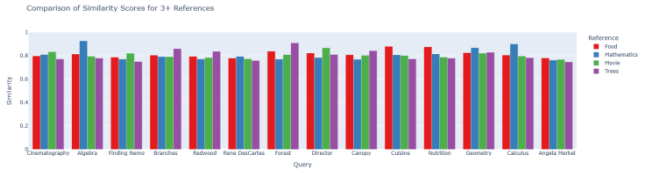

Fig 4: Performance of text-embedding-ada-002 while comparing words

Analyzing the graphs, it is evident that text-embedding-large-3 and text-embedding-small-3 can capture semantic relationships better than the older model text-embedding-ada-002. Upon careful analysis it can be observed that, text-embedding-large-3 gives more accurate similarity scores and can-do better differentiate between relevant and irrelevant references. This suggests that text-embedding-large-3 provides superior contextual understanding compared to other earlier models, making it the most effective choice for word and phrase-level comparisons.

The same model was also found to be the most effective for document-to-word and document-to-document similarity comparisons. The detailed graphical results supporting these findings can be accessed in Listing 1.

Listing 1: Complete Similarity Analysis Results
The full set of similarity comparison graphs, supporting the findings of this paper—including the selection of the best embedding model and the most effective similarity metric for all modes of comparison, and the results from the exponentially weighted mean of similarities—is available in the dataset. The Excel file containing all results can be accessed at the following link: Similarity Analysis Graphs (Excel File)

B. Comparison of Similarity Metrics

To evaluate the effectiveness of different similarity metrics, they were analyzed across various text comparison scenarios. The goal was to determine which metric provides the most meaningful differentiation between text pairs at different levels of comparison.

Fig. 5, 6, 7, and 8 illustrate the performance of Cosine Similarity, Dot Product, Euclidean Distance, and Jaccard Similarity respectively when used for comparing a set of query and reference words or phrases. The embedding model used was text-embedding-large-3.

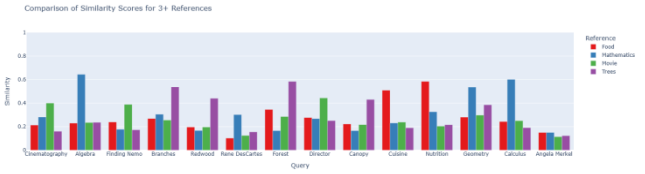

Fig 5: Performance of Cosine similarity while comparing words

Fig 6: Performance of Dot Product similarity while comparing words

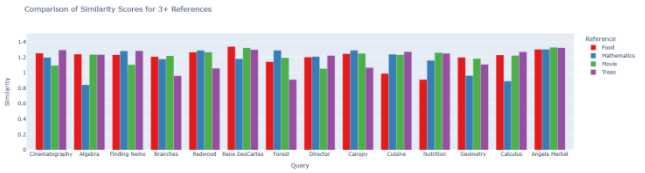

Fig 7: Performance of Euclidean Distance while comparing words

Fig 8: Performance of Jaccard similarity while comparing words

Euclidean distance measures the absolute distance between two vectors in the embedding space, with smaller distances indicating higher similarity. However, as shown in Fig 7, Euclidean distance struggles to effectively differentiate between texts compared to cosine and dot product similarity. Additionally, its interpretation is less intuitive, making it less suitable for practical semantic similarity analysis.

Jaccard similarity quantifies the overlap between two sets of elements, making it well-suited for Bag-of-Words-based comparison methods rather than embedding-based similarity analysis. Since Jaccard similarity computes similarity by counting common scalar values in the embedding vectors, it fails to capture semantic relationships effectively, as demonstrated in Fig 8.

Figs 5 and 6 demonstrate that cosine similarity and dot product similarity yield identical results. This occurs because OpenAI's embeddings are unit-normalized, ensuring that their magnitudes do not influence similarity calculations. While both metrics outperform Jaccard and Euclidean similarity in accuracy, cosine similarity is chosen as the primary similarity metric for this application.

Although dot product is computationally efficient, cosine similarity is preferred because it focuses solely on the relative alignment of vectors, whereas dot product similarity is influenced by vector magnitudes. This distinction is crucial for ensuring robustness and adaptability. If non-normalized embedding models are used in the future, dot product similarity may produce inconsistent results, whereas cosine similarity remains reliable. Thus, selecting cosine similarity enhances the application's flexibility for integration with different embedding models.

C. *Performance of Mode 1: Comparing Words/Phrases with Words/Phrases*

To measure the performance while comparing words or phrases with words/phrases, similarity analysis of a dataset with 52 query texts including words and phrases and four reference text was used as input. The first 11 rows of the similarity table generated in presented in Table II.

TABLE II. COMPARISON RESULTS OF WORDS WITH WORDS

Q\R	Science & Technology	Politics & Leadership	Arts & Entertainment	Sports
Karl Marx	0.105	0.197	0.114	0.138
Winston Churchill	0.143	0.285	0.112	0.141
Serena Williams	0.105	0.119	0.177	0.289
Alan Turing	0.226	0.105	0.091	0.106
Nikola Tesla	0.259	0.094	0.139	0.126
Jeff Bezos	0.18	0.208	0.153	0.131
Michael Phelps	0.105	0.151	0.127	0.282
William Shakespeare	0.14	0.153	0.205	0.163
Roger Federer	0.099	0.124	0.161	0.251
Actor Vijay	0.109	0.098	0.177	0.153

The analysis revealed that the classification results were highly accurate for internationally recognized and well-documented personalities. However, the model struggled with correctly categorizing regionally famous individuals. Additionally, the embeddings were not always up to date. For instance, the south Indian actor Vijay, who recently transitioned into politics, was primarily associated with the entertainment domain, with no significant relation to politics. These limitations highlight the dependency on the training data, which may not always reflect recent developments.

D. *Performance of Mode 2: Comparing Documents with Words/Phrases and the Impact of Exponentially Weighted Mean Similarity*

The effectiveness of the Exponentially Weighted Mean Similarity method in improving word/phrase-to-document comparisons is demonstrated using a real-world use case.

Case Study: Identifying Relevant Research Papers

Consider two research papers:
Document 1: *Multicultural Experiences: A Systematic and New Theoretical Framework* – A 32-page study examining how multicultural experiences impact psychological, interpersonal, and organizational outcomes.
Document 2: *Teamwork in a Multicultural Context: The Driver of Creativity* – A 5-page paper discussing how cultural factors influence team creativity.

The objective is to determine which document contains relevant information about the impact of cultural diversity on individuals, using the application. This information is present in Document 1. To evaluate this, the query documents are set as Document 1 and Document 2, and the reference phrase is "Impact of working in a multicultural team on individuals".

a) Comparison Using Standard Mean Similarity: When computing the simple mean of similarities between the phrase and document chunks, the results (Fig 9) fail to distinguish the correct document. Instead, it falsely identifies document 2 to have the relevant data. This method is only able to capture the overall context.

b) Comparison Using Exponentially Weighted Mean Similarity: Applying the Exponentially Weighted Mean Similarity method, the results (Fig 10) successfully differentiate between the two documents, correctly identifying Document 1 as the one containing the most relevant information. This improvement is due to the method's ability to assign higher weights to more relevant chunks, thereby overcoming the limitations of a simple average similarity score.

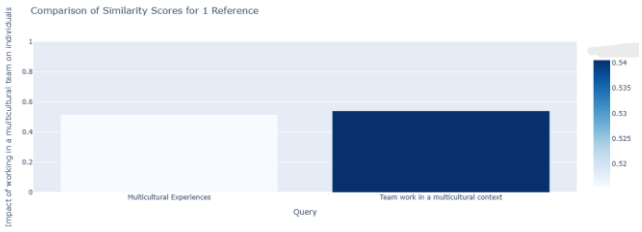

Fig 9: Result while using simple mean for similarities

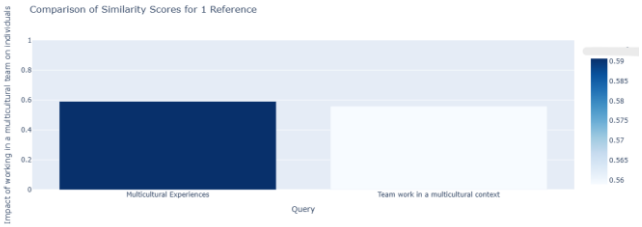

Fig 10: Result while using exponentially weighted mean for similarities

E. Visualization of Results

To enhance the interpretation of similarity results, a Dash-based web application was developed, providing interactive visualizations for analyzing query-reference relationships. The application reads the similarity table generated by the system and presents the data using bar graphs, scatter plots, and three-dimensional graphs, adapting the visualization format based on the number of reference texts:

- Single reference → Bar graph (Fig. 11)
- Two references → 2D scatter plot (Fig. 12)
- Three references → 3D scatter plot (Fig. 13)
- Four or more references → Bar graph (Fig. 14)

These visualizations provide an intuitive way to identify patterns and trends in similarity scores, making it easier to

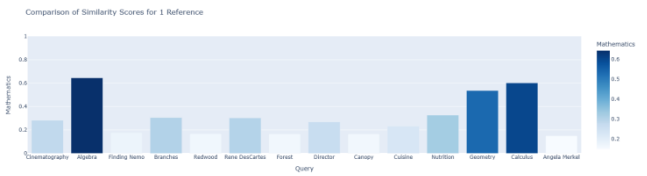

Fig 11: Visualization using bar graph when there is only one reference

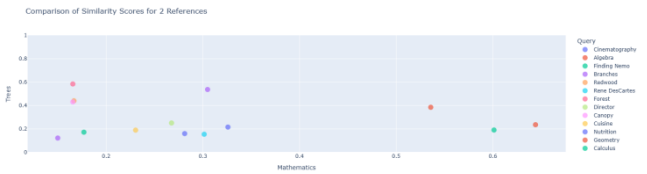

Fig 12: Visualization using scatter plot when there are two references

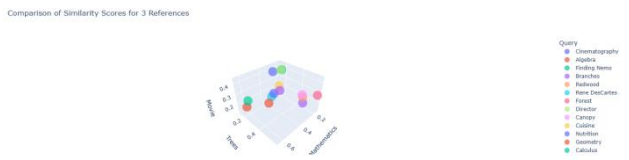

Fig 13: Visualization using scatter plot when there are three references

Fig 14: Visualization using bar graphs when there are more than three references

determine which reference text is most relevant to a given query.

In addition to similarity score visualization, the system includes a one-dimensional representation of embedding vectors, offering further insights into their structure (figure. 15 and 16). The scalar values of each embedding vector are plotted as a one-dimensional line graph, where the x-axis corresponds to the index of the embedding dimensions (0 to 3072 for text-embedding-3-large) and the y-axis represents the corresponding scalar values, ranging from -1 to +1. This visualization provides a deeper understanding of how textual data is numerically represented within the embedding space and highlights variations in the encoded representations of words, phrases, and documents.

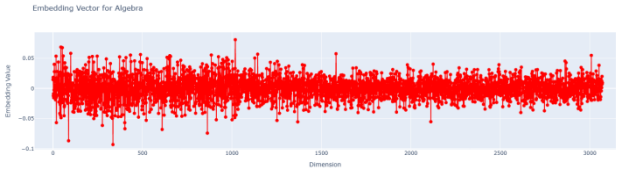

Fig 15: Scalar values of an embedding vector

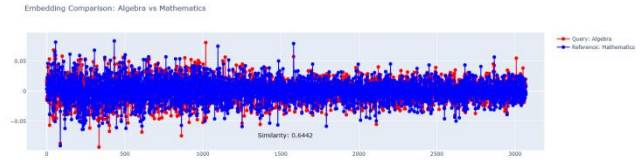

Fig 16: Scalar values overlap of two embedding vectors being compared with similarity score

Comprehensive unit testing was conducted for all major methods to ensure the reliability of the system (Fig 17).

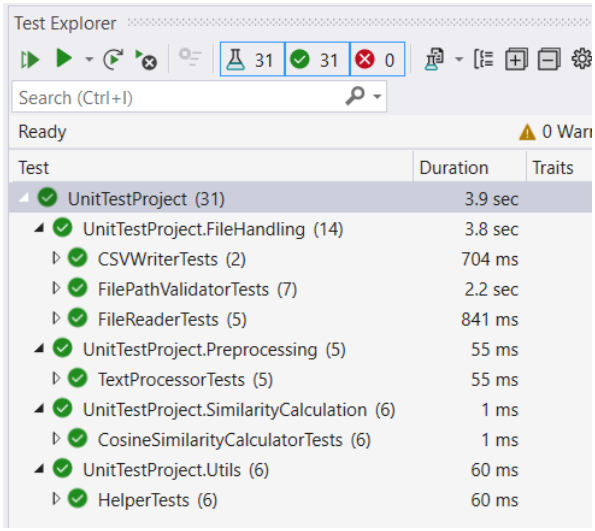

Fig 17: Unit Tests

IV. Discussion

This study introduced a systematic approach for multi-level text similarity analysis by leveraging OpenAI's embedding models and various similarity metrics. A comprehensive evaluation was conducted to determine the most effective embedding model and similarity metric for different types of comparisons, including word-to-word, word-to-document, and document-to-document similarity. The results demonstrated that *text-embedding-3-large* consistently outperformed other models, and cosine similarity was identified as the most reliable metric due to its robustness across different embedding scales.

To improve fine-grained similarity analysis, an exponentially weighted mean of similarities was implemented, effectively distinguishing relevant documents in word-to-document comparisons. Additionally, an interactive Dash-based web application was developed to visualize scalar value oof embedding vectors using line plots and similarity relationships using bar graphs, scatter plots, and three-dimensional plots, aiding in result interpretation. However, the approach comes with increased computational cost and processing time due to the increased number of embeddings, which may slow down the application, when used for comparing big documents on a larger scale. It was also observed that the accuracy of the similarity results depends upon the quality and scope of training data used to train the embedding model.

Future improvements to this work can include enhancing performance through the implementation of multithreading, which can significantly reduce processing time for large datasets by parallelizing embedding generation and similarity calculations. Additionally, the application has the potential to be refined for specific domains by customizing preprocessing techniques and fine-tuning the β value in the exponentially weighted mean function. When customizing for a specific domain, document chunking can be more optimized by incorporating advanced semantic chunking methods. This can enhance accuracy by dynamically adjusting chunk sizes based on content structure and optimize performance. Furthermore, expanding the application to support newer embedding models will provide greater flexibility and potentially improve performance in various text similarity tasks. These enhancements will contribute to making the system more scalable, efficient, and adaptable for real-world applications across different fields, such as academic research, legal document analysis, and content recommendation systems.

References

[1] H. Cao, “Recent advances in text embedding: A comprehensive review of top-performing methods on the MTEB benchmark,” *arXiv preprint*, arXiv:2406.01607, 2024. Available: https://arxiv.org/abs/2406.01607.

[2] S. Zhang, Y. Liang, M. Gong, D. Jiang, and N. Duan, “Multi-view document representation learning for open-domain dense retrieval,” *arXiv preprint*, vol. 2203.08372, 2022. Available: https://arxiv.org/abs/2203.08372.